

Anteater Poker Game (Version 2.0)

Software Specification

By All In

Members: Sanaa Bebal, Ava Chinn, Alonso De La Cruz, Estrella Distancia Zamora, Lauren Howe, Samuel Kim, and David Lee

Created for the class EECS 22L at University of California, Irvine

Table of Contents

Table of Contents	2
Glossary	3
Game	3
Betting Stages in a Hand (In Chronological Order)	4
Hand Rankings	4
Moves	5
1. Software Architecture Overview	5
1.1 Main Data Types and Structures	5
1.2 Major Software Components	7
1.3 Module Interfaces	7
1.4 Overall Program Control Flow	12
1.5 Automated Client: Poker Bot	12
2. Installation	13
2.1 System requirements and compatibility	13
2.2 Setup and configuration	13
2.3 Building, compilation, and installation	14
3. Documentation of Packages, Modules, and Interfaces	14
3.1 Detailed Description of Data Structures	14
3.2 Detailed Description of Functions and Parameters	15
3.3 Detailed Description of Communication Protocol	21
4. Development Timeline	22
4.1 Partitioning of Tasks	22
4.2 Team Member Responsibilities	22
5. References and External Libraries	23
Contact Information	23
Terms and Conditions	23
Copyright	23
Disclaimer of Warranty	23
Index	24

Glossary

(of terms used in the implementation)

Game

- **Anteater Poker**—a version of poker based on Texas Hold'em, with the addition of the anteater cards and their rules (see “anteater card”)
- **Anteater card**—two exist in a deck; not a part of any standard suit; only a ranked hand when a player holds two anteaters in the hole cards, in which case it ranks between a royal and straight flush and four of a kind
- **Board**—a term used to refer to the visible community cards
- **Blinds**—the bids made by the player one left (the small blind) and two left (the big blind) of the dealer; small blind is 1 point and big blind is 2 points
- **Community cards**—the cards visible to all players that can be used by all players to create a five-card hand
- **Dealer**—the player in charge of giving cards to players; dealer deals clockwise
- **Five-card hand**—a group of five cards comprised of some combination of the two hole cards and the five community cards
- **Hand**—refers to the complete sequence of moves made within the repeatable segment of the game, starting with players receiving their cards and ending with someone winning the hand
- **Hole cards**—the two cards held by the individual player; only the player knows what their cards are and only the player can use them to create a five-card hand
- **Kicker**—refers to the cards that are not part of the ranked part of a hand that are used to break ties; used for ties involving one pair, two pair, three of a kind, and four of a kind
- **Pot**—holds the main points bet in a hand
- **Round**—consists of dealing cards and multiple betting intervals where the players match the highest bet, raise, or fold. Rounds include pre-flop, flop, turn, and river betting.
- **Side pot**—not the main pot, but a pot used when at least one player goes all in and the other players continue betting; cannot be won by the player(s) who went all in
- **Suit**—a group of thirteen cards with values ranging from two to ace; the four types ranked lowest to highest are clubs, diamonds, hearts, and spades
- **Value**—refers to the second attribute in addition to suit that a card has; the fourteen values are 2-10, Jack, Queen, King, Ace, and Anteater
- **Game end**—game automatically ends when one player wins all the points or when game is ended via server

Betting Stages in a Hand (In Chronological Order)

1. **Preflop**—no cards on the board; betting starts with the small blind and the big blind, then proceeds to the player left of the big blind; each player can call, raise, or fold until all players have either met the highest bet made or folded
 - a. Note: During this stage, the only player who can check is the player who made the big blind, and this only occurs when everyone else has called or folded (i.e. not raised); the big blind player can then choose to either check (ending the preflop round) or raise (continuing the round, as all players then have to call, raise, or fold)
2. **Flop**—three cards on the board; each player can check (if no one else has bet), bet, call, raise, or fold until all players have either met the highest bet made or folded
3. **Turn** - After the flop, the dealer reveals the fourth community card; often where draws are complete and the pot size increases dramatically to force decisions
4. **River** - The last community cards is dealt in a game and where the final betting round begins
5. **Showdown** - the final stage of a hand in which all betting has concluded and only occurs when there are at least two players remaining after the final betting round or checking on the river. Typically, the last player to make a bet or raise on the final round is the first to reveal their hand, but if someone checks, the player to the left of the dealer reveals their hand first.

Note: If at any point all but one player has folded in the rounds leading up to the showdown, the one player left wins the pot.

Hand Rankings

1. **Royal flush**—the player has the 10, Jack, Queen, King, and Ace of all one suit
2. **Straight flush**—the player has five cards of consecutive value all of the same suit (i.e. a straight all in the same suit)
3. **Anteater Pair**—the player holds two anteaters in the hole cards
4. **Four of a kind**—the player has four cards of the same rank and one of another rank
5. **Full house** - the hand has three cards of one rank and two cards of another rank
6. **Flush** - a hand contains five cards of sequential order, not necessarily in consecutive order
7. **Straight** - a five-card hand containing sequential ranks, not considering the suits
8. **Three of a kind** - consisting of three cards of the same rank with two unmatched cards
9. **Two pair** - contains two cards of one rank, two cards another rank, and one card of a third rank
10. **Pair** - consists of two cards of the rank and three unrelated or unmatched cards
11. **High card** - contains none of the hands above, the hand is determined by the highest ranking card

The royal flush is the highest-ranking hand possible in the game

Moves

- **All-in**– Wager all of a player's remaining points on a single hand. The player can't take any more betting actions unless they win additional points in any side pots.
- **Bet**– The first voluntary wage made in a betting round.
- **Call**– Match the current highest bet made by the previous player. Only available when a preceding player has made a bet or raise during a betting round.
- **Check**– Pass the action to the next player without wagering. Only available if no player has bet before you in that round.
- **Fold**– Discarding your current hand and forfeit any points contributed in the pot
- **Raise**– Increase the current highest bet, requiring other players to match the new amount to keep their hand. Only available after a bet has been made.

1. Software Architecture Overview

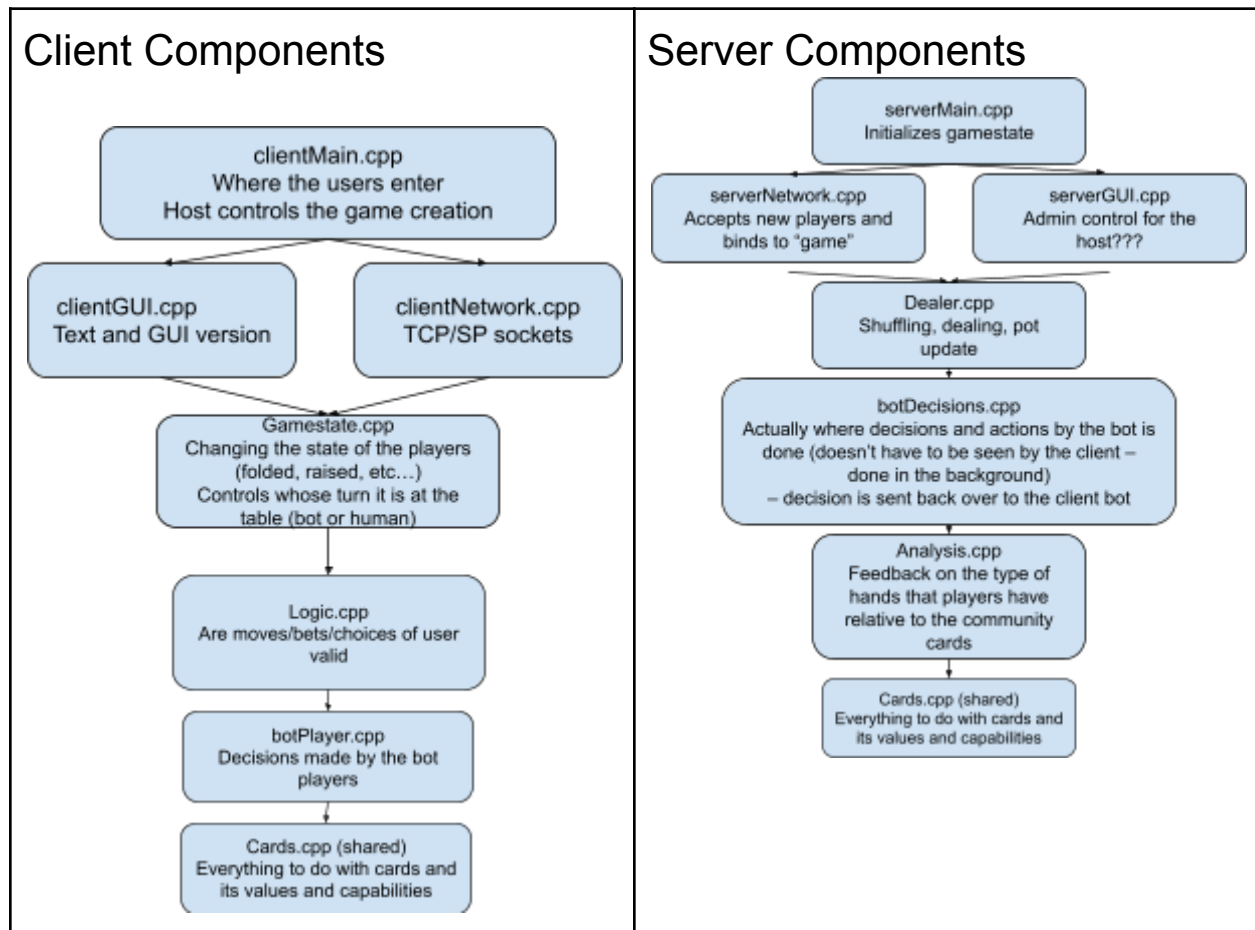
1.1 Main Data Types and Structures

- Enumerators:
 - enum suit–represents Hearts, Clubs, Diamonds, Spades, and Anteaters
 - enum val–represents Two-Ten, Jack, Queen, King, Ace, and Anteater
 - enum Round–represents Preflop, Flop, Turn, River, and Showdown
 - enum PlayerType–represent Human (1) and Computer (2) (players)
 - enum SpecialBets–assigning Check to 0 and Fold to -1
 - enum PackageType–delineates between the types of data sent between the client and server (login and game)
 - **Note:** Although not an enum, player numbering starts at 0
- Structures:
 - struct gameState
 - int packageType = game // used for sending data via the buffer between the client and the server (see PackageType enum)
 - int numPlayers // holds the number of players in the game
 - int round // holds the round number (Preflop, Flop, Turn, River, or Showdown)
 - int dealerPlayer // holds which player is the “dealer” (note that dealing is done by the server, not the player)
 - PILES allCards // holds all the player hands, the community cards, and the leftover cards (in that order)
 - int pot // represents the amount of money in the pot

- int callAmount // the amount of money needed to call in the round
 - PLAYERS players // holds player information
 - int playerTurn // holds which player's turn it is
 - int greatest // holds which player has made the biggest bet so far
 - struct Player
 - int playerNum // holds player number
 - char *name // holds the player's name
 - int playerType // holds if player is Human (1) or Computer (2) (see PlayerType enum)
 - int score // holds a player's current score (does not include points that have been bet during the hand)
 - int bet // holds a player's bet for the current round of betting
 - int isInHand // still playing in the hand or not (i.e. 1 if they haven't folded, 0 if they have)
 - int isEliminated // should be removed from the list if this is true
 - int playerSocket // this struct can be removed if struct Player is only used by the server & int playerSocket can be moved back into the struct
 - struct LoginInfo
 - int packageType = login // used for sending data via the buffer between the client and the server (see PackageType enum)
 - int isHost // represents if the player is the "host" player (the host determines the number of players to enter the game, the number of bots, etc.)
 - char *playerName // holds the player's name
 - int playerNum // represents the player number at the table (numbering starts at zero)
 - char *password // set by the player if the host (from client to server), or entered by a joining player if not the host (from server to client)
 - int playerType (Human or Computer (see PlayerType enum))
 - int numPlayers // number of players in the room
 - PLAYERSFOUND playersFound // 0 if player is not found; 1 or 2 if player is Human / Computer (see PlayerType enum)
 - typedef struct LoginInfo LOGININFO;
 - typedef struct playerState GAMESTATE;
 - typedef struct Player PLAYER;
- Lists (vectors):
 - typedef std::vector<PLAYER> PLAYERS; // used to hold info about the players
 - typedef std::vector<Card> PILE; // used to hold one of these items: deck, player hands, community cards, leftover cards, etc.
 - typedef std::vector<PILE> PILES; // used to hold player hands, community cards, and leftover cards (in that order)
 - typedef std::vector<unsigned char> BUF; // used to represent the buffer for TCP/IP communication
 - typedef std::vector<int> PLAYERSFOUND; // used in struct LoginInfo (see above)
- Classes:

- Card class
 - Attributes: val, suit
 - Methods:
 - Card() and Card(int valVar, int suitVar) [constructors]
 - getValue() and getSuit() [getters]

1.2 Major Software Components



1.3 Module Interfaces

Major functions in each module

clientGUI.cpp

Local files required: `Gamestate.hpp`, `Analysis.hpp`, `clientGUI.hpp`

Libraries required: <gtk/gtk.h> <string.h> <stdio.h> <vector>

Internal Functions:

void updateHostPlayerList(const vector<PLAYER>& players, int maxPlayers)

- Calls: none

void updateJoinPlayerList(const vector<PLAYER>& players, int maxPlayers)

- Calls: none

void setAvailableSlots(const vector<int>& slots)

- Calls: none

void updateGameState(const vector<PLAYER>& players, const vector<Card>& community,
const vector<Card>& hole, int pot, int currentBet, int localStack, int playerTurn, int dealerTurn)

- Calls: none

void setGameActions(bool enabled)

- Calls: none

void setResults(const vector<PLAYER>& results, const GAMESTATE& summary)

- Calls: none

gamestate.cpp

Local files required: gamestate.hpp, cards.hpp, data.hpp, scoreCalc.hpp

Libraries required: <stdio.h>, <stdlib.h>, <vector>, <string.h>, <time.h>

Internal Functions:

- PILE oneShuffle(PILE deck)
 - Calls:
- PILE shuffle(PILE deck)
 - Calls:
- PILES deal(PILE deck, int numPlayers, int dealerPlayer)
 - Calls:
- GAMESTATE updateGameState(GAMESTATE recvGameState)
 - Calls:
- GAMESTATE startRound(GAMESTATE gameState)
 - Calls: none
- GAMESTATE endRoundChecks(GAMESTATE gameState)
 - Calls: none
- GAMESTATE preflopUpdate(GAMESTATE gameState)
 - Calls: none
- GAMESTATE flopUpdate(GAMESTATE gameState)
 - Calls: none
- GAMESTATE turnUpdate(GAMESTATE gameState)
 - Calls: none
- GAMESTATE riverUpdate(GAMESTATE gameState)
 - Calls: none
- GAMESTATE showdownUpdate(GAMESTATE gameState)
 - Calls: none

gameOverScreen.cpp

Local files required: gameOverScreen.hpp

Libraries required: <sstream>, <cmath>

Internal Functions:

- void setResults(const vector<PLAYER>& results, const GAMESTATE& summary)
 - Calls: N/A
- static gboolean onDrawHand(GtkWidget* widget, cairo_t* cr, gpointer data)
- static void onPlayAgainClicked(GtkButton* button, gpointer data)
 - Calls: static_cast<gameOverScreen*>(data)
- static void onExitLobbyClicked(GtkButton* button, gpointer data)
 - Calls: static_cast<gameOverScreen*>(data)

hostScreen.cpp

Provides: the screen the the player hosting the game

Requires:

- void playerList(const vector<PLAYER>& players, int maxplayers)
 - Calls: N/A

Cards.cpp (Reshuffles, reinitializes cards in deck; shared between client and server)

Local files required: Cards.hpp, Constants.hpp

Libraries required: <vector>, <algorithm>, <cstdlib>, <stdio.h>, <stdlib.h>

Internal Functions:

- PILE initDeck(void)
 - Calls: N/A
- void printCard(Card card)
 - Calls: N/A
- int printDeck(PILE deck)
 - Calls: N/A
- int printPiles(PILES piles)
 - Calls: printDeck()

serverGUI.cpp (displaying basic game/player info on the server end)

Local files required: Cards.hpp, Gamestate.hpp

Libraries required: <gtk/gtk.h> <string.h> <stdio.h> <vector>

Internal functions:

- void ShutdownClicked(GtkWidget *Widget, gpointer Data)
 - Calls:
- void UpdatePlayer(LOGININFO loginInfo, int playerSocket)
 - Calls:
- void UpdateScores(GAMESTATE gameState)

- Calls:
- void UpdateWindow(void)
 - Calls:
- void FatalError(const char *ErrorMsg)
 - Calls: none
- int MakeServerSocket(uint16_t PortNo)
 - Calls: none
- void MessageAllClients()
 - Calls: none
- void ProcessClientRequest(int DataSocketFD)
 - Calls: none
- void ProcessLoginRequest(int DataSocketFD, BUF RecvBuf)
 - Calls: createBuffer(), parsingLoginArguments()
- void ProcessGameStateRequest(int DataSocketFD, RecvBuf)
 - Calls: createBuffer(), parsingGameArguments()
- void ServerMainLoop(int ServSocketFD, int Timeout)
 - Calls: ProcessClientRequest()

botDecisions.cpp (where decisions and actions by the bot is done)

Local files required: cards.hpp, botDecisions.hpp, data.hpp, scoreCalc.hpp

Libraries required: <stdio.h>, <stdlib.h>, <vector>, <random>

Internal functions:

- int decideAction(GAMESTATE gameState, int playerNum)
 - Calls: calculateOdds(), validCall(), validCheck(), validRaise(), bluffing()
- int getRaiseAmount(GAMESTATE gameState, int playerNum)
 - Calls: none
- double calculatedOdds(GAMESTATE gameState, int playerNum)
 - Calls: calculateOdds(), bluffing()

scoreCalc.cpp (Calculates the score of each active player's hands)

Local files required: cards.hpp, data.hpp, scoreCalc.hpp

Libraries required: <stdio.h>, <stdlib.h>, <iostream>, <unordered_map>, <vector>

Internal Functions:

- int scoreHand(PILES piles, int playerNum)
 - Calls: N/A
- int isRoyalFlush(PILES piles, int playerNum)
 - Calls: N/A
- int isStraightFlush(PILES piles, int playerNum)
 - Calls: N/A
- int isAnteaterPair(PILES piles, int playerNum)
 - Calls: N/A
- int isFourKind(PILES piles, int playerNum)
 - Calls: kicker()

- `int isFullHouse(PILES piles, int playerNum)`
 - Calls: N/A
- `int isFlush(PILES piles, int playerNum)`
 - Calls: N/A
- `int isStraight(PILES piles, int playerNum)`
 - Calls: N/A
- `int isThreeKind(PILES piles, int playerNum)`
 - Calls: N/A
- `int isTwoPair(PILES piles, int playerNum)`
 - Calls: N/A
- `int isOnePair(PILES piles, int playerNum)`
 - Calls: N/A
- `int isHighCard(PILES piles, int playerNum)`
 - Calls: N/A
- `int val_translator(int x)`
 - Calls: N/A
- `int makeScore (int hand, int max, int second, int third, int fourth, int fifth)`
 - Calls: N/A

DataTransfer.cpp (Helps transfer data between client and server)

Local files required: `cards.hpp`, `data.hpp`

Libraries required: `<stdio.h>`, `<vector>`, `<string.h>`, `<assert.h>`

Internal Functions:

- `BUF createBuffer(GAMESTATE gameState)`
 - Calls: N/A
- `BUF createBuffer(LOGININFO LoginInfo)`
 - Calls: N/A
- `GAMESTATE parsingGameArguments(BUF &buf)`
 - Calls: N/A
- `LOGININFO parsingLoginArguments(BUF &buf)`
 - Calls: N/A

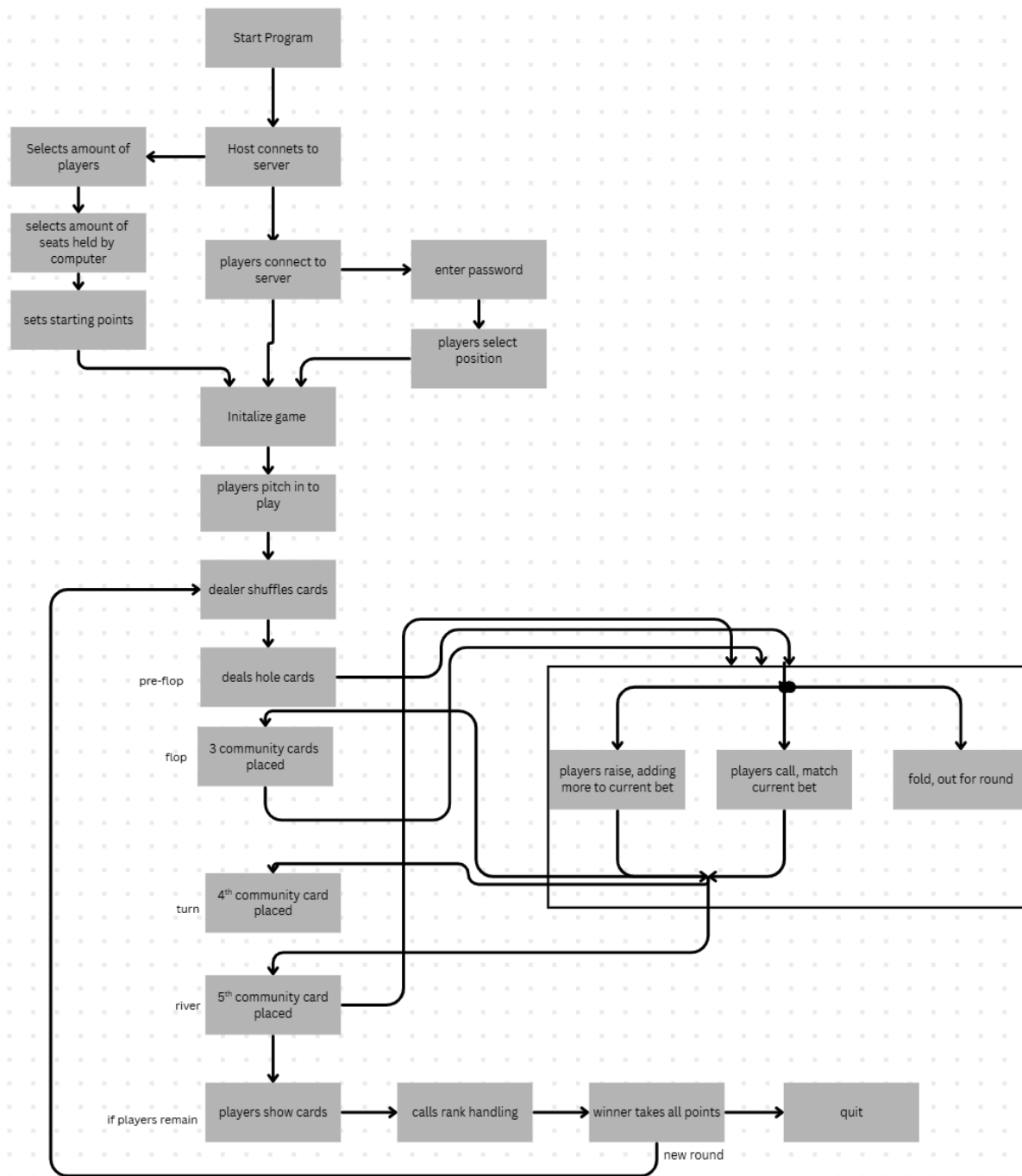
JoinScreen.cpp

Local files required: `data.hpp`, `cards.hpp`

Libraries required: `<gtk/gtk.h>`, `<functional>`, `<vector>`, `<string>`,

- `void playerList(const vector<PLAYER>& players, int maxPlayers)`
 - Calls: N/A
- `void openSlots(const vector<int>& slots)`
 - Calls: N/A

1.4 Overall Program Control Flow



1.5 Automated Client: Poker Bot

For our automated client, the bot players and all decision-making will occur on the server side, in the `botDecisions.cpp`, while the client side, in the `botDecisions.cpp` receives the bot's actions and then applies them to the client display or application. The bot itself remains as a separate client process that connects to the game server, similarly to a human-controlled client.

In general, the automated player is just another participant in the poker game, replacing a human user. Whenever there are extra seats to fill, or a host versus computer, depending on the host user's specifications for the current game, the bot players are easily identifiable through the `PlayerType` enum, which distinguishes between human and computer players within the shared game state.

The server-side controls all bot decision-making: based on the current game state, the bot evaluates it, and a legal action is then decided for the selected bot player. Relevant functions like `decideAction()`, `evaluateHand()`, and `calculateOdds()` all support the bot's actions, such as check, call, raise, or fold. From there, the server sends the selected action to the bot's client application, which then processes it like another human client would.

Since each bot operates as its own client, communication with the server must be maintained to update information and respond only when it is the given bot player's turn. The design is intended to keep the bot separate and allow for both the human and computer to coexist in the game environment without excessive special-case handling.

2. Installation

2.1 System requirements and compatibility

The user must satisfy the following conditions to correctly install the software:

- **Operating System:** RHEL8 64-bit Linux
- **Compiler:** GNU Compiler Collection (gcc)
- **Build System:** GNU *make*
- **Libraries:** Standard C libraries; GTK 3.0
- **Utilities:** tar, gzip, evince, and shell environment
- **Hardware Requirements:** Minimal CPU and memory resources sufficient for compiling and executing.
- **Server Access:** Must be able to access the UCI EECS Servers

2.2 Setup and configuration

Using the command line, go into the installed folder and run the following code:

```
A. Extract the Source Package
gtar xvzf Poker_V1.0_src.tar.gz
cd poker
```

B. Build the Software

make (from within the poker directory)

C. Run the Executable

bin/poker (from within the poker directory)

D. Remove compiled object files

make clean (from within the poker directory)

2.3 Building, compilation, and installation

A. Create the Binary Distribution

```
tar -zcvf Poker_V1.0.tar.gz \  
    ./doc/README \  
    ./doc/COPYRIGHT \  
    ./doc/Poker_UserManual.pdf \  
    ./doc/INSTALL \  
    ./bin/poker \  
    ./bin/*.bmp \  
    ./Makefile
```

B. Create the Source Distribution

```
tar -zcvf Poker_V1.0_src.tar.gz \  
    ./doc \  
    ./bin \  
    ./src \  
    ./Makefile
```

C. Purpose of Each Package

Poker_V1.0.tar.gz - Contains the executable and documentation for end users

Poker_V1.0_src.tar.gz - Contains full source code for developers and maintainers

3. Documentation of Packages, Modules, and Interfaces

3.1 Detailed Description of Data Structures

Detailed descriptions of enumerators, typedefs, and structures can be found in Section 1.1.

Their actual code will be similar to their setup within this section.

The Card class, however, is included below because its details and setup go beyond what was explained in Section 1.1.

Card class (cards.hpp):

```
class Card{
public:
    int val;
    int suit;

    Card(void) { // overloading
        val = -1;
        suit = -1;
    }
    Card(int valVar, int suitVar){
        val = valVar;
        suit = suitVar;
    }

    int getValue(void) const { // const means funct can't modify
anything--good for all getters
        return val;
    }
    int getSuit(void) const{
        return suit;
    }
};
```

3.2 Detailed Description of Functions and Parameters

Note: There will also be unit and system testing files in the final version. For brevity purposes, they are not included in these descriptions.

clientGUI.cpp -

Provides: text and GUI application

Requires: stdio.h, stdlib.h, unistd.h, string.h, netinet/in.h, netdb.h, gtk/gtk.h, assert.h

void updateHostPlayerList(const vector<PLAYER>& players, int maxPlayers)

- Arguments: vector of connected players, max players allowed
- Returns: N/A

void updateJoinPlayerList(const vector<PLAYER>& players, int maxPlayers)

- Arguments: vector of connected players, max players allowed
- Returns: N/A

void setAvailableSlots(const vector<int>& slots)

- Arguments: vector of open seats
- Returns: N/A

void updateGameState(const vector<PLAYER>& players, const vector<Card>& community, const vector<Card>& hole, int pot, int currentBet, int localStack, int playerTurn, int dealerTurn)

- Arguments: active players, community cards, players cards, total point in pot, amount needed to call current bet, remaining points for player, players turn, dealer turn
- Returns: N/A

void setGameActions(bool enabled)

- Arguments: bool flag
- Returns: N/A

void setResults(const vector<PLAYER>& results, const GAMESTATE& summary)

- Arguments: vector of players with final hand rankings, final game state
- Returns: N/A

gameState.cpp -

Provides: controls the next actions of the game

Requires:

PILE oneShuffle(PILE deck)

- Arguments: vector containing the 54 cards initially
- Result: shuffles the deck once

PILE shuffle(PILE deck)

- Arguments: vector containing the 54 cards initially
- Result: shuffles deck

PILES deal(PILE deck, int numPlayers, int dealerPlayer)

- Arguments: vector containing the 54 cards initially,
- Result: deals the cards to the players

GAMESTATE updateGameState(GAMESTATE recvGameState)

- Arguments: specifies the state of the game
- Result: updates the state of the current game

GAMESTATE startRound(GAMESTATE gameState)

- Arguments: GAMESTATE structure specifying the state of the game
- Result: starts the round

GAMESTATE endRoundChecks(GAMESTATE gameState)

- Arguments: GAMESTATE structure specifying the state of the game
- Result: checks everything after the end of a round

GAMESTATE preflopUpdate(GAMESTATE gameState)

- Arguments: GAMESTATE structure specifying the state of the game
- Result: updates after preflop

GAMESTATE flopUpdate(GAMESTATE gameState)

- Arguments: GAMESTATE structure specifying the state of the game
- Result: updates after flop

GAMESTATE turnUpdate(GAMESTATE gameState)

- Arguments: GAMESTATE structure specifying the state of the game
- Result: updates after turn

GAMESTATE riverUpdate(GAMESTATE gameState)

- Arguments: GAMESTATE structure specifying the state of the game
- Result: updates after river

GAMESTATE showdownUpdate(GAMESTATE gameState)

- Arguments: GAMESTATE structure specifying the state of the game
- Result: updates after showdown

gameOverScreen.cpp -

Provides: screen for when the game ends

Requires:

void setResults(const vector<PLAYER>& results, const GAMESTATE& summary)

- Arguments: final player standings, game over summary
- Result: N/A

static gboolean onDrawHand(GtkWidget* widget, cairo_t* cr, gpointer data)

- Arguments: drawing widget, cairo graphics, user pointer data
- Result: returns if the card was successfully handled

static void onPlayAgainClicked(GtkButton* button, gpointer data)

- Arguments: click restart button widget, user pointer data
- Result: resets table state

static void onExitLobbyClicked(GtkButton* button, gpointer data)

- Arguments: click restart button widget, user pointer data
- Result: disconnects player from game session

hostScreen.cpp-

Provides: the screen the the player hosting the game

Requires:

void playerList(const vector<PLAYER>& players, int maxplayers)

- Arguments: current players, max number of players
- Results: formats and prints list of players

cards.cpp - initialization of cards in the deck and shared between client and server

Provides: all the values for each of the specific cards in the deck

Requires:

PILE initDeck(void)

- Arguments: None
- Result: returns an initialized deck

void printCard(Card card)

- Arguments: None

- Result: Prints a card based on its value and suit

int printDeck(PILE deck)

- Arguments: PILE vector containing a list of cards
- Result: can be used to print any PILE, including player hands and the board

int printPiles(PILES piles)

- Arguments: PILE vector containing all of the cards
- Result: returns back the location of cards whether in the players hand, community deck, and in the cards to be drawn

serverGUI.cpp - displaying basic game/player info on the server end

Provides: a display to the host user for relevant game information

Requires:

void ShutdownClicked(GtkWidget *Widget, gpointer Data)

- Arguments: gtk button widget, pointer to user data
- Result: cleanup and applies application closure

void UpdatePlayer(LOGININFO loginInfo, int playerSocket)

- Arguments: account and profile information of the player, socket file associated with the player
- Results: updates player GUI

void UpdateScores(GAMESTATE gameState)

- Arguments: current game state
- Results: updates score display

void UpdateWindow(void)

- Arguments: none
- Results: updates server's GUI

void FatalError(const char *ErrorMsg)

- Arguments: pointer to why the program cannot run
- Results: none

int MakeServerSocket(uint16_t PortNo)

- Arguments: host port number
- Results: initialized socket

void MessageAllClients()

- Arguments: none
- Results: sends message to connected players

void ProcessClientRequest(int DataSocketFD)

- Arguments: target client network communication socket
- Results: none

void ProcessLoginRequest(int DataSocketFD, BUF RecvBuf)

- Arguments: target client network communication socket, vector containing login credentials
- Results: none

void ProcessGameStateRequest(int DataSocketFD, RecvBuf)

- Arguments: target client network communication socket , vector containing player move entries
- Results: none

void ServerMainLoop(int ServSocketFD, int Timeout)

- Arguments: master network for incoming player connections, time duration
- Results: none

botDecisions.cpp - where decisions and actions by the bot is done

Provides: all logic needed for the bot players to make decisions (all settled actions will be sent to the client to be completed)

Requires:

int decideAction(GAMESTATE gameState, int playerNum)

- Arguments: makes final decision of what move to make
- Result: returns a number that reflects the player's action (bet, call, fold, etc.)

int getRaiseAmount(PILES piles, int playerNum)

- Arguments:
- Result: returns back scored based upon the community deck

double calculateOdds(int handScore)

- Arguments: determines likelihood of winning based on current hand and flop
- Result: returns back percentage of likelihood of winning

scoreCalc.cpp - Calculates score for each active player's hands

Provides: Scoring for hands (based on showdown round—i.e. all cards visible)

Requires:

int scoreHand(PILES piles, int playerNum)

- Arguments: list of player hands, the board cards, and the leftover cards (PILES vector); player number
- Result: the best score for each

int isRoyalFlush(PILES piles, int playerNum)

- Arguments: list of player hands, the board cards, and the leftover cards (PILES vector); player number
- Result: returns 11000 if possible and 0 if not

int isStraightFlush(PILES piles, int playerNum)

- Arguments: list of player hands, the board cards, and the leftover cards (PILES vector); player number
- Result: returns 10,000 base score + high card tiebreaker bonus if possible and 0 if not

int isAnteaterPair(PILES piles, int playerNum)

- Arguments: list of player hands, the board cards, and the leftover cards (PILES vector); player number
- Result: return 9,000 if possible and 0 if not

int isFourkind(PILES piles, int playerNum)

- Arguments: list of player hands, the board cards, and the leftover cards (PILES vector); player number
- Result: Returns an int in (1000, 2000] for a pair, in (3000,4000] for 3-of-a-kind, and in (7000,8000] for 4-of-a-kind; score is base score + tiebreaker bonus

int isFullHouse(PILES piles, int playerNum)

- Arguments: list of player hands, the board cards, and the leftover cards (PILES vector); player number
- Result: returns an int in (6000,7000] if there is a full house and 0 if not (score is base score + bonus determined by values within the ranked hand for tiebreaking purposes)

int isFlush(PILES piles, int playerNum)

- Arguments: list of player hands, the board cards, and the leftover cards (PILES vector); player number
- Result: returns an int in (5000,6000] if there is a flush and 0 if not (score is base score + tiebreaker bonus)

int isStraight(PILES piles, int playerNum)

- Arguments: list of player hands, the board cards, and the leftover cards (PILES vector); player number
- Result: returns an int in (4000,5000] if there is a straight and 0 if not (score is base score + tiebreaker bonus)

int isThreeKind(PILES piles, int playerNum)

- Arguments: list of player hands, the board cards, and the leftover cards (PILES vector); player number
- Result: score for three of a kind hand

int isTwoPair(PILES piles, int playerNum)

- Arguments: list of player hands, the board cards, and the leftover cards (PILES vector); player number
- Result: returns an int in (2000,3000] if there is a two pair and 0 if not (score is base score + tiebreaker bonus)

int isHighCard(PILES piles, int playerNum)

- Arguments: list of player hands, the board cards, and the leftover cards (PILES vector); player number
- Result: returns an int within (0,1000] if there is a high card and 0 if not (score is base score + tiebreaker bonus)

int val_translator(int x)

- Arguments: none
- Result: returns a value depending on the card

int makeScore(int hand, int max, int second, int third, int fourth, int fifth)

- Arguments:
- Results: makes the score for each hand

DataTransfer- where data transfer between the server and client happens

Provides: the necessary information for data transfer between client and server

Requires:

BUF createBuffer(GAMESTATE& gameState)

- Arguments: GAMESTATE structure specifying the state of the game
- Results: creates a buffer struct for the gamestate

BUF createBuffer(LOGININFO LoginInfo)

- Arguments: struct holding user authentication data
- Results: creates a buffer struct for the login credentials

GAMESTATE parsingGameArguments(BUF &buf)

- Arguments: reference to a buffer received from the network
- Results: reconstructed gamestate struct

LOGININFO parsingLoginArguments(BUF &buf)

- Arguments: reference to a buffer containing login data
- Results: reconstructed login info struct

JoinScreen.cpp - GUI for players joining game

Requires:

void playerList(const vector<PLAYER>& players, int maxPlayers)

- Arguments: current player, number of max players
- Results: formats and displays list of active players

void openSlots(const vector<int>& slots)

- Arguments: id's of empty slots
- Results: open slot number so user can choose a set

3.3 Detailed Description of Communication Protocol

The poker game uses a client-server model in which all player actions/decisions are sent from the specific client to the server. All game-state updates and next steps stem from the server back to the client. The server will be in charge of all game flow, card dealing, pot updates, turn order, and round transitions, so that clients can easily display information and send necessary input. This helps to prevent clients from changing game state and keeping the game flow synchronized.

Function calls

client functions

- socket(), gethostbyname(), connect(), bzero(), read()

server functions

- socket(), bzero(), htons(), bind(), listen(), accept(), read()

Communication will be separated into login and game data. The login messages are primarily for the first player or host of the game, while in comparison game messages are for active play to transmit updates concerning player turns, bets, community cards, and round changes.

Protocol flags and settings

- packageType for both gameState and LoginInfo so the receiver identifies incoming buffers
- LoginInfo includes isHost, playerType, numPlayers, playerNum, password, & playersFound

4. Development Timeline

4.1 Partitioning of Tasks

Week 6	Week 7	Week 8	Week 9	Week 10
Create user manual	Create software specification Divide work amongst team members	Create alpha version of code with text-based interface Focus on relaying messages to clients via the server	Create beta version of code Fix any bugs with alpha version	Create final version of the code

4.2 Team Member Responsibilities

Sanaa Bebal - clientGUI.cpp / serverGUI.cpp / clientMain.cpp

Ava Chinn - logic.cpp / cards.cpp / botDecisions.cpp / botPlayer.cpp

Alonso De La Cruz - logic.cpp / cards.cpp / botDecisions.cpp / botPlayer.cpp/ documentation

Estrella Distancia - clientMain.cpp / serverMain.cpp / serverGUI.cpp/ card design

Lauren Howe - / gameState.cpp / cards.cpp / analysis.cpp

Samuel Kim - logic.cpp / gameState.cpp/ scoreCalc.cpp

David Lee - gameState.cpp / logic.cpp / scoreCalc.cpp

5. References and External Libraries

<https://canvas.eee.uci.edu/courses/81045/pages/projects>

Used for specification of the rules and requirements of Anteater Poker

https://www.linuxhowtos.org/C_C++/socket.htm

Used as a reference for serverNetwork.cpp

<https://github.com/jmqueen01/EECS22L-chess/blob/main/README>

Used as a reference for installation Linux code

Contact Information

If you have any questions, comments, or concerns about this software, please contact the creators using the following email addresses:

- sbebal@uci.edu
- amchinn@uci.edu
- alonsojd@uci.edu
- edistanc@uci.edu
- lehowe@uci.edu
- samuek16@uci.edu
- davijl18@uci.edu

Note: Any email pertaining to this software sent to one or more of the addresses above may be forwarded to any or all of the other emails above.

Terms and Conditions

Copyright

© 2026 All In. All rights reserved.

The unauthorized sale or distribution of this software is prohibited.

Disclaimer of Warranty

THIS SOFTWARE IS PROVIDED “AS IS” WITHOUT ANY EXPRESS OR IMPLIED WARRANTY OF ANY KIND.

Index

All in - 5
Analysis.cpp - 12, 20
Anteater card - 3
Anteater Pair - 4
Anteater Poker- 3
Bet - 5
Blinds - 3
Board -3
botDecisions.cpp - 11, 19
botPlayer.cpp - 9, 18
Call - 5
Cards.cpp- 9, 19-20
Check - 5
Classes - 6
clientGUI.cpp - 9, 16
clientMain.cpp - 9, 16
Community cards - 3
Dealer.cpp - 10, 19
Dealer - 3
Five-card hand - 3

Flop - 4
Flush - 4
Fold - 5
Four of a kind - 4
Full house - 4
Game end - 3
Gamestate.cpp - 9, 17
Hand - 3
High card - 4
Hole cards - 3
Kicker - 3
Logic.cpp - 9, 17
Pair - 4
Pot - 3
Preflop - 4
Raise - 5

River - 4
Round - 3
Royal flush - 4
serverGUI.cpp - 10, 19

severMain.cpp - 10, 19
serverNetwork.cpp - 10, 20
Showdown - 4
Side pot - 3
Straight - 4
Straight flush - 4
struct gameState - 5
struct loginInfo
struct Player - 6
Suit - 3
Three of a kind - 4
Turn - 4
Two pair - 4
Value - 3